

Pac-Man : un terrain de mesure pour l'intelligence artificielle

Documentation technique du projet

Module IA_PO : Projet d'Intelligence Artificielle Libre, Master 1, 2025-2026

Dépôt : <https://github.com/Marwwannn/Pacman>

Support oral en ligne :

https://docs.google.com/presentation/d/1ykfCRB4OQPf8w0sW2Sskm844TaiuC9IRnrmg_pgL1Q/edit?usp=sharing

1. Contexte et besoin

1.1 Le problème

Comparer des approches d'intelligence artificielle demande un terrain qui ne triche pas. Les environnements tout faits (Gym, ALE) sont pratiques mais opaques : on y mesure un agent sans jamais pouvoir expliquer ce que l'environnement lui a fait subir. Quand un agent stagne, on ne sait pas distinguer « *la méthode est mauvaise* » de « *le problème est mal posé* ».

Ce projet prend le parti inverse : **construire le terrain et les agents, et les mesurer ensemble**. Le moteur de jeu est écrit ligne à ligne, donc chaque propriété dont l'apprentissage dépend (déterminisme, vitesse, topologie) est connue, mesurée, et testée.

1.2 Pourquoi Pac-Man

Pac-Man est un banc d'essai classique de l'IA, et pour de bonnes raisons :

- **Les règles sont simples, la stratégie ne l'est pas.** Il faut arbitrer en permanence entre le gain (les pastilles) et le risque (les fantômes), sur un horizon long.
- **L'adversaire est intelligent mais imparfait.** Les fantômes de 1980 suivent une règle myope : à chaque case, ils prennent la direction qui réduit la distance à vol d'oiseau vers leur cible, sans voir les murs. C'est cette myopie qui les rend battables, et qui rend le jeu apprenable.
- **Le score est une mesure honnête.** Pas de jugement humain à interpréter.

1.3 Public visé

Qui veut voir un agent apprendre sur un problème qu'il comprend entièrement : enseignement, démonstration, base de comparaison pour d'autres méthodes. Le jeu étant jouable dans un navigateur, le

comportement de l'IA est lisible sans lire une ligne de code : on *voit* l'agent hésiter à une intersection.

2. Fonctionnalités

2.1 Le jeu

- Moteur déterministe complet : labyrinthe, pastilles, super-pastilles, huit fruits bonus, vies, niveaux, chaîne de points sur les fantômes mangés, vie supplémentaire à 10 000 points, tunnels latéraux.
- Les **quatre personnalités de fantômes** de l'arcade, y compris le bug d'adressage de 1980 (cible décalée quand Pac-Man regarde vers le haut), reproduit et désactivable.
- Alternance des modes *scatter* / *chase* par vagues, mode effrayé, retour à la maison des fantômes mangés.
- Format de labyrinthe en texte : ajouter un plan ne demande aucun code.

2.2 Les interfaces

- **Client web** servi par le même serveur : rendu canvas, sprites vectoriels, interpolation du mouvement, HUD, classement, bruitages synthétisés, clavier et tactile. Aucune dépendance, aucune étape de build.
- **API REST** documentée automatiquement (`/docs`) : créer une partie, envoyer une direction, avancer d'un tick, lire l'état, gérer les scores.
- **WebSocket** : le serveur fait tourner la partie à 60 ticks/s et diffuse une image par tick.
- **CLI** `pacman-r1` : `baselines` , `train` , `compare` .

Regarder l'IA jouer. Le client a un mode spectateur : `?ia=appris` fait jouer le modèle final par le serveur, dans le même jeu, à la même cadence ; les trois autres agents sont disponibles de la même façon. Les touches de direction sont ignorées, la pause reste au spectateur, et le score de l'IA n'entre pas dans le classement. Le pilote côté serveur reprend la discipline de l'environnement d'entraînement, et un test vérifie case par case que la partie jouée est celle que l'environnement aurait jouée. Pour le rejeu commenté, décision par décision, voir §5.8.

2.3 Les intelligences artificielles

	NATURE	APPREND ?	RÔLE
4 fantômes	règles, fidèles à 1980	non	l'adversaire
Joueur aléatoire	tirage uniforme	non	plancher de comparaison
Joueur heuristique	règles écrites à la main	non	plafond raisonnable
Joueur Q approximé	Q-learning linéaire	oui	le cœur du projet
Joueur de recherche	simulation en avant	non	ce que coûte de ne pas apprendre

Le dernier mérite un mot. Les trois premiers décident à partir de ce qu'ils **voient** ; celui-ci décide à partir de ce qui **arriverait** : à chaque intersection il clone la partie, joue chaque coup possible, laisse le moteur dérouler la suite, et garde la meilleure issue.

Le moteur étant déterministe, cette recherche est **exacte** : il n'y a ni nœud de hasard ni espérance à estimer, ce qui la distingue d'un expectimax ou d'un MCTS classiques. C'est une propriété du terrain, pas de l'agent.

3. Choix techniques

3.1 Bibliothèques

Le principe directeur : **aucune dépendance dans le cœur métier**.

DÉPENDANCE	OÙ	POURQUOI
(aucune)	core/ , ai/ , rl/	testable seul, portable, rien à installer pour apprendre
FastAPI + Uvicorn	api/	validation des entrées et documentation OpenAPI gratuites
(aucune)	web/	modules ES natifs : pas de build, pas de <code>node_modules</code>
pytest, ruff	tests	test et style

Ce qui a été volontairement écarté : NumPy (douze poids ne justifient pas un tableau), PyTorch (voir §3.4), Gymnasium (l'interface `reset / step` tient en trente lignes, l'importer aurait ajouté une dépendance lourde pour une convention de nommage), et tout moteur de jeu (le rendu est du canvas).

3.2 Architecture

```
src/pacman/  
├─ core/      modèle du jeu : ne connaît ni le réseau ni FastAPI  
├─ ai/        comportement des fantômes (hérite de core.Ghost)  
├─ rl/        agent joueur par apprentissage  
├─ api/       REST + WebSocket  
├─ mazes/     labyrinthes au format texte  
└─ web/       client (HTML, CSS, 10 modules ES)
```

Le sens des dépendances est strict et à sens unique : `web` → `api` → `core`, et `ai`, `rl` → `core`. Le cœur ne sait pas qu'il est exposé par un serveur, ce qui permet de le faire tourner 3 000 fois plus vite que le temps réel pour l'apprentissage, sans modifier une ligne.

Héritage et polymorphisme structurent le modèle :

```
Entity (ABC)  
├─ Pacman  
└─ Ghost  
    └─ PersonalityGhost      (ai/) : une cible, une personnalité  
        └─ Blinky            vise Pac-Man  
        └─ LookaheadGhost    vise devant Pac-Man  
            └─ Pinky          quatre cases devant  
                └─ Inky       symétrise Blinky par rapport à Pinky  
            └─ Clyde          fuit vers son coin sous huit cases
```

Les quatre fantômes ne diffèrent **que par la case visée** : la méthode `target()` est redéfinie, tout le reste (déplacement, modes, collisions) est hérité. C'est ce qui rend fidèle une IA de 204 lignes.

Côté joueurs, le polymorphisme passe par un `Protocol` (typage structurel) plutôt que par une classe de base : les trois agents n'ont rien à partager en implémentation, seulement un contrat `act(game, actions, env) -> Direction`. L'environnement, l'évaluation et le CLI les traitent indifféremment.

3.3 Les deux décisions qui portent tout le projet

① Une décision par intersection, pas par tick.

Mesure faite avant d'écrire l'agent : sur les 300 cases praticables du labyrinthe classique, **34 seulement offrent un vrai choix** (degré ≥ 3). 89 % du plan est un couloir où la direction est imposée. Décider à chaque tick produirait environ 2 440 décisions sans conséquence par partie.

L'environnement traverse donc les couloirs lui-même et ne rend la main qu'aux intersections. **L'horizon tombe de ~2 500 pas à ~100** : le problème d'attribution du crédit (« lequel de mes coups m'a tué ? ») devient 25 fois plus court, sans rien perdre du jeu. C'est le gain le moins cher du projet.

② Le déterminisme du moteur est traité comme un piège.

Le moteur est parfaitement reproductible : deux parties lancées avec le même état donnent le même score au tick près. C'est indispensable pour tester, et c'est un piège pour apprendre. Même labyrinthe, mêmes fantômes, même départ : un agent y mémorise **une suite de coups** qui donne un score flatteur à l'entraînement et s'effondre au moindre changement. On croirait avoir appris une politique, on aurait appris un itinéraire.

Parade appliquée à trois niveaux :

1. chaque épisode tire une graine qui décale le départ de Pac-Man et l'errance des fantômes ;
2. l'évaluation se fait sur une **plage de graines disjointe** de l'entraînement : `evaluate()` lève une erreur si on lui passe une graine d'entraînement, ce n'est pas une convention mais une garde ;
3. les résultats se lisent en **médiane et écart-type sur 100 parties**, jamais au meilleur run.

3.4 Pourquoi le Q-learning approximé

Le non-supervisé strict (clustering, autoencodeur) a été écarté au cadrage : il construit une *représentation*, jamais une *politique*. Il ne peut pas décider. Restait à choisir la forme de l'apprentissage par renforcement.

APPROCHE	VERDICT	MOTIF
Q tabulaire	exclu	l'état brut se compte en 2^{244} rien que pour les pastilles : aucune généralisation possible
Q approximé linéaire	retenu	12 poids, quelques minutes d'entraînement, et des poids lisibles
DQN (réseau profond)	exclu	10 à 100 × plus d'épisodes, une dépendance PyTorch, et une boîte noire à défendre à l'oral

La lisibilité a pesé lourd : $Q(s, a) = w \cdot f(s, a)$ sur douze descripteurs nommés signifie qu'après entraînement, on lit ce que l'agent a compris (§5.3). Un réseau de neurones aurait peut-être marqué plus de points, sans rien apprendre à personne.

3.5 Les douze descripteurs

Toutes les features sont bornées dans $[0, 1]$, et ce n'est pas cosmétique : avec des amplitudes hétérogènes, un même taux d'apprentissage ferait diverger un poids pendant qu'un autre bougerait à peine.

Les distances sont **réelles dans le labyrinthe** (parcours en largeur avec cache), jamais à vol d'oiseau : deux cases séparées par un mur épais sont proches en ligne droite et très loin dans les faits. C'est précisément l'erreur que commettent les fantômes de 1980 : l'agent, lui, ne la commet pas.

FEATURE	CE QU'ELLE DIT
biais	constante, capte la valeur moyenne d'un coup
mange_pastille , mange_super_pastille , mange_fruit	la case visée porte-t-elle un gain immédiat
proximite_pastille , proximite_super_pastille	à quelle distance est le prochain gain
proximite_chasseur , chasseurs_proches	le danger, en distance et en nombre
proximite_proie	opportunité : un fantôme effrayé est à +200 minimum
issues	combien de sorties offre la case visée (une impasse en offre zéro)
demi_tour	l'action est-elle un retour en arrière
avancement	part de pastilles déjà mangées : sépare le début et la fin de partie

3.6 Aucune information du futur

Un agent est facile à rendre bon par accident : il suffit qu'il voie les fantômes **un tick en avance**. Le score monte, et il ne prouve plus rien.

Garantie tenue ici : les fantômes sont lus à la position qu'ils occupent **au moment du choix**. La seule projection est la case où Pac-Man arriverait, et c'est l'effet de l'action évaluée, donc précisément ce qui rend le descripteur dépendant de l'action, pas un renseignement que le jeu refuse au joueur.

Ce n'est pas une affirmation, c'est une mesure. Le test se place sur un tick où un fantôme **bouge réellement**, calcule les deux proximités possibles (depuis sa position d'avant, depuis celle d'après), vérifie qu'elles diffèrent, puis vérifie que l'agent lit bien la seconde. Sans le contrôle « elles diffèrent », le test passerait aussi avec une lecture décalée d'un tick.

Deux autres gardes complètent : évaluer une action ne fait **pas avancer la partie** (un extracteur qui simulerait un tick pour « voir venir » serait détecté), et déplacer un fantôme change **immédiatement** ce que l'agent voit : aucune mémoire, aucune anticipation.

L'exception est assumée et déclarée : l'agent de recherche (§2.3) simule l'avenir. C'est sa définition même, et c'est pourquoi il gagne : voir §5.8.

3.7 Deux jeux de descripteurs, et pourquoi les comparer

Le jeu ci-dessus ne décrit les fantômes que par des **agrégats** : le chasseur *le plus proche*, la pastille *la plus proche*. Deux fantômes à huit cases y sont donc indiscernables d'un seul. D'où une question qui se tranche par la mesure, pas par l'intuition : **donner à l'agent la position de chaque fantôme et la répartition de la nourriture le rend-il meilleur ?**

Un second jeu, `positions` (26 poids), l'a été ajouté pour le savoir :

AJOUT	CONTENU
3 nombres × 4 fantômes	sa distance, si l'action m'en rapproche , s'il est comestible
2 nombres pour la nourriture	direction de la masse de pastilles, densité locale

Ces positions sont exprimées dans le repère de Pac-Man, jamais en coordonnées absolues. Ce n'est pas un détail de forme : dans un modèle linéaire, un poids sur `x` signifierait « préfère la droite du plan » : une règle qui ne généralise à rien. Une distance et un « ça me rapproche » sont la même information, exprimée là où elle est apprenable.

Les fantômes sont rangés du plus proche au plus lointain. Sans cet ordre, échanger deux fantômes identiques changerait le vecteur, et l'agent devrait apprendre quatre fois la même chose.

Le jeu de base **reste** et sert de témoin : `scripts/comparer_descripteurs.py` oppose les deux à graines, hyperparamètres et curriculum identiques. Résultat au §5.5.

3.8 Le barème de récompenses

Le barème pèse plus lourd sur le résultat final que α et γ réunis.

ÉVÉNEMENT	VALEUR	POURQUOI CETTE VALEUR
Pastille	+10	
Super-pastille	+50	
Fantôme mangé	+200 ... +1600	chaîne du jeu d'origine
Fruit	valeur du niveau	
Mort	-500	doit dominer : sinon l'agent se suicide pour abrégé une partie coûteuse
Pas de décision	-1	indispensable : sinon il tourne devant une super-pastille sans jamais la manger, le <i>reward hacking</i> classique de Pac-Man
Niveau terminé	+500	

Le score brut du jeu **n'est pas repris comme récompense** : la vie supplémentaire à 10 000 points y crée une marche que rien dans l'état ne permet de prédire, donc du bruit pur pour l'apprentissage.

3.9 La part de l'IA dans le dépôt

L'énoncé demande que l'IA représente « à minima 70 % du projet ». Compté en lignes, ce dépôt ne l'atteint pas, et il vaut mieux le dire que le laisser compter : les modules d'IA (`ai/` , `r1/` et les quatre scripts de mesure) font **47 % du Python**, 36 % avec le client web. Le reste est le moteur et l'API. Mais ils n'ont pas été écrits pour eux-mêmes : le moteur a été conçu comme un **banc d'essai** (déterministe, clonable, sans horloge) et c'est ce qui a permis le protocole d'évaluation (§5), la recherche en ligne (§5.3) et le rejeu de chaque décision (§5.8). Sans terrain mesurable, aucun agent n'est évaluable. C'est la lecture que ce document défend ; le chiffre brut est là pour que le lecteur puisse ne pas la partager.

4. Validation et tests

```
pytest                                # 278 tests, ~1 min
pytest --cov=pacman --cov-report=term  # 97 % de couverture
ruff check src tests
```

Les mêmes commandes tournent en intégration continue à chaque push (`.github/workflows/tests.yml` , Python 3.11 et 3.13) : un dépôt dont les tests ne passent que sur la machine de son auteur n'est pas validé.

Les tests ne se contentent pas de vérifier que le code s'exécute, ils **mesurent les propriétés dont tout le reste dépend** :

- deux parties de même graine donnent le même score au tick près ;
- l'évaluation refuse les graines d'entraînement ;
- chaque pas de l'agent se termine sur une vraie intersection ;
- l'agent ressort seul d'un cul-de-sac.

Ce dernier test a une histoire instructive : il itérait sur une liste vide et passait donc systématiquement, **parce que le labyrinthe classique ne contient aucune impasse**. Il a été scindé en deux : un test qui *mesure* cette absence, et un test du cul-de-sac joué sur un plan 11×7 construit exprès. Un test vert qui ne prouve rien est pire qu'un test absent : il rassure.

5. Métriques

(voir `results/campagne.json` : chiffres reproduits par `python scripts/campagne_rl.py`)

Toutes les mesures ci-dessous viennent d'une seule commande (`python scripts/campagne_rl.py`), sur **100 parties par agent**, à $\epsilon = 0$, sur des graines de la plage d'évaluation, jamais vues pendant les 3000 épisodes d'entraînement. Elles sont reproductibles à l'identique.

Ces tableaux sont générés par `scripts/injecter_resultats.py` depuis `results/campagne.json` : aucun chiffre n'est recopié à la main.

5.1 Un fantôme : l'agent atteint-il le plafond ?

AGENT	SCORE MÉDIAN	ÉCART-TYPE	MIN	MAX	VICTOIRES	MORTS
aleatoire	605	374	0	1590	0%	100%
heuristique	2870	895	50	3500	35%	65%
q-approxime	2730	405	1030	3390	51%	49%

5.2 Quatre fantômes : le jeu complet

AGENT	SCORE MÉDIAN	ÉCART-TYPE	MIN	MAX	VICTOIRES	MORTS
aleatoire	470	335	0	1700	0%	100%
heuristique	2340	1952	50	10330	0%	100%
q-approxime	2895	1171	560	5800	4%	96%
recherche	4990	1815	0	10690	94%	7%

Et le témoin, entraîné directement à quatre fantômes sans passer par un :

AGENT	SCORE MÉDIAN	ÉCART-TYPE	MIN	MAX	VICTOIRES	MORTS
q-approxime	2915	1136	340	5820	0%	100%

Le curriculum **n'améliore donc pas le score** : il n'apporte qu'un peu de régularité et quelques victoires. La recommandation initiale était bonne comme méthode (elle sépare « l'agent n'apprend pas » de « le problème est trop dur ») ; elle ne l'était pas comme gain de performance, et c'est la mesure qui le dit.

5.3 Jusqu'où faut-il chercher ?

La recherche n'a qu'un réglage : la profondeur, en points de décision. Son coût suit, et il se paie à chaque coup joué.

PROFONDEUR	SCORE MÉDIAN	ÉCART-TYPE	VICTOIRES	MORTS
1	3255	1128	34%	66%
2	4290	1241	83%	17%
3	4990	1815	94%	7%

5.4 Ce que l'agent a appris, poids par poids

Les douze poids du modèle entraîné à quatre fantômes, du plus fort au plus faible. C'est l'intérêt d'un modèle linéaire : on lit la politique.

DESCRIPTEUR	POIDS	LECTURE
biais	+1071.77	constante, sans effet sur les décisions (identique pour toutes les actions)
avancement	-910.87	prudence croissante en fin de partie
proximite_proie	+461.08	chasse les fantomes effrayes
proximite_chasseur	-380.27	fuit les chasseurs
proximite_super_pastille	+299.39	garde les super-pastilles a portee
proximite_pastille	+199.22	se rapproche des pastilles
chasseurs_proches	-199.20	fuit l'encerclement
mange_pastille	-111.86	signe trompeur : colineaire avec proximite_pastille , qui vaut 1 sur la meme case
mange_super_pastille	-90.62	se mefie de la super-pastille
demi_tour	-49.96	evite le demi-tour
issues	-1.02	quasi nul : n'a rien tranche
mange_fruit	+0.00	va chercher le fruit

Comment lire ces signes. Deux precautions, sans quoi deux lignes de ce tableau se lisent a l'envers :

- Le **biais** est identique pour toutes les actions : il ne departage rien. Sa valeur mesure le retour moyen d'une partie, pas une preference.
- Deux descripteurs peuvent etre **colineaires**. Sur une case qui porte une pastille, `mange_pastille` vaut 1 et `proximite_pastille` vaut 1 : les deux poids se partagent le credit, et seule leur **somme** a un sens. Ici elle reste positive : l'agent va bien manger.

Ce que le tableau dit vraiment : l'agent a appris la **peur avant la gourmandise**. Fuir les chasseurs et l'encerclement pese plus lourd que n'importe quelle pastille, et chasser une proie effrayee pese plus encore. Ce sont exactement les trois priorites de l'heuristique ecrite a la main, sauf que personne ne les lui a dites.

5.5 La courbe d'apprentissage (1 fantôme)

Score médian par fenêtre de 500 épisodes, pendant l'entraînement, donc avec l'exploration encore active, ce qui explique qu'il reste sous le score final.

ÉPISODE	E	SCORE MÉDIAN
500	0.77	770
1000	0.55	1215
1500	0.32	1885
2000	0.10	2390
2500	0.05	2650
3000	0.05	2620

5.6 Les positions aident-elles ?

L'expérience a été **demandée par l'auteur** (14/08) : « donne à l'agent la position de chaque fantôme et de la nourriture ». Même curriculum, mêmes graines, mêmes hyperparamètres : seul le jeu de descripteurs change.

DESCRIPTEURS	POIDS	MÉDIANE 1F	MÉDIANE 4F	ÉCART-TYPE 4F	VICTOIRES 4F	ENTRAÎNEMENT
base	12	2730	2900	1216	1%	988 s
positions	26	2580	2390	1073	0%	1153 s

Les positions n'apportent pas : -510 points (-18%). Deux fois plus de poids à estimer sur le même budget d'épisodes, pour une information que les agrégats portaient déjà en grande partie. Le coût d'entraînement, lui, est mesuré : $\times 1.2$.

C'est le genre de résultat qu'un projet honnête doit publier tel quel. La question « faut-il donner plus d'information à l'agent ? » n'a pas de réponse évidente : plus de descripteurs, c'est plus de poids à estimer sur le même budget d'épisodes, et un modèle linéaire ne peut de toute façon pas composer ces informations entre elles.

5.7 Les fantômes portaient toujours des mêmes cases

Un angle mort du protocole, trouvé sur **une question de l'auteur** en relecture (« les positions des fantômes sont aléatoires ? ») à laquelle la réponse a été cherchée en **comptant** ce que les graines faisaient varier, au lieu de relire le code censé le produire : sur 100 parties, l'évaluation ne produisait **1 seule configuration de départ des fantômes**. La graine ne resyait que la case de Pac-Man et l'errance en mode effrayé. La garde sur les graines protège donc contre la mémorisation d'une *partie*, pas contre la dépendance à une *configuration*, et rien dans les chiffres précédents ne permettait de trancher.

Le test est court et **ne réentraîne rien** : les mêmes poids sont réévalués avec les quatre fantômes tirés au sort hors de la maison (100 configurations sur 100 parties). Cette condition ne joue pas la même partie : les quatre fantômes y sont actifs dès le premier tick. On l'attendait plus dure ; elle est en réalité plus **facile**, parce que dispersés ils partent chacun vers son coin au lieu de sortir groupés du centre. C'est exactement pourquoi les agents qui n'ont rien appris servent de témoins : ils absorbent la variation de difficulté, quel qu'en soit le sens.

AGENT	RÉFÉRENCE	DISPERSÉS	ÉCART	IC 95 %	
aleatoire	470	340	-130	[-260, +15]	dans le bruit
heuristique	2340	2585	+245	[-380, +965]	dans le bruit
appris	2895	3025	+130	[-300, +440]	dans le bruit
recherche	4990	5890	+900	[+455, +1400]	significatif

L'agent **appris** tient : +130 points, intervalle [-300, +440] : il contient zéro, donc l'écart est indiscernable du bruit d'échantillonnage. Sa politique ne dépend pas de la maison centrale : elle est **topologique**, comme ses descripteurs le laissaient espérer sans que personne ne l'ait vérifié. C'était la seule façon de le savoir.

Trois des quatre écarts sont du bruit. Le seul qui sorte est celui de la **recherche** (+900 points, intervalle [+455, +1400]), et il s'explique : elle replanifie depuis l'état réel à chaque intersection, donc des chasseurs étalés dans le labyrinthe lui sont franchement plus simples qu'une vague sortant du centre.

Une nuance à ne pas cacher : le taux de victoire de l'agent appris passe de 4% à 0%. Sur 100 parties cela représente une poignée de parties, trop peu pour en conclure quoi que ce soit, mais assez pour ne pas prétendre que la condition dispersée lui est en tout point équivalente.

(mesure : `python scripts/fantomes_ailleurs.py` → `results/fantomes_ailleurs.json`, intervalles par bootstrap sur 2000 tirages, graine fixe)

5.8 Voir l'agent décider

Les poids du §5.4 disent ce que l'agent a appris *en moyenne*. Ils ne disent pas pourquoi il a tourné à gauche à la trente-deuxième intersection.

```
python scripts/exporter_decisions.py    # puis ouvrir docs/decisions.html
```

La page produite rejoue une partie et rend **chaque décision lisible** : le labyrinthe à cet instant, les directions envisagées, ce que chacune valait, et la décomposition terme à terme qui a tranché : en vert ce qui pousse à y aller, en rouge ce qui retient.

Elle met aussi en évidence quelque chose qu'aucun tableau de poids ne montre : un descripteur **identique pour toutes les directions** ne choisit rien, quel que soit son poids. Le biais est dans ce cas par construction, et `avancement` aussi : il décrit l'état, pas le coup. Sur les douze descripteurs, deux ne participent donc jamais à un arbitrage. Ils sont sortis du chiffre affiché.

C'est le seul des quatre agents dont on puisse ouvrir la décision de cette façon, et c'est exactement ce qui a été acheté en refusant le réseau de neurones (§3.4).

5.9 Apprendre ou recalculer

L'agent de recherche ne sait rien, n'a rien appris, n'a aucun poids, et il gagne largement (\$5.2). Le résultat n'est pas décevant pour l'apprentissage : il est la mesure de ce que l'apprentissage achète.

	Q APPROXIMÉ	RECHERCHE
Coût avant de jouer	3 000 épisodes, ~8 min	zéro
Coût pendant le jeu	12 multiplications par coup	des dizaines de parties simulées par coup
Ce qu'il reste après	12 nombres, transférables	rien
Si le moteur devient indisponible ou coûteux	fonctionne toujours	s'effondre
Si les règles changent	à réentraîner	s'adapte seul

La recherche est meilleure ici parce qu'elle a accès à un simulateur exact et gratuit. C'est un luxe : dans presque toutes les applications réelles (un robot, un marché, un patient), simuler l'avenir est soit impossible, soit plus cher que d'agir. L'apprentissage existe précisément pour ces cas-là.

Autrement dit, ce comparatif ne dit pas « la recherche gagne », il dit « sur un terrain où l'on peut tout simuler, il ne faut pas apprendre », et la vraie question devient : dispose-t-on d'un tel terrain ?

6. Usage de l'IA générative

Cette section est une déclaration, exigée par l'énoncé. Elle est reprise et détaillée dans le README (section « Usage IA »).

6.1 Ce qui a été utilisé

Claude Code (Anthropic, modèles Claude Opus 4.8 puis Claude Opus 5 : le modèle exact figure dans le trailer de chaque commit), en ligne de commande, du 19/07/2026 au 21/08/2026. Aucun autre outil d'IA générative.

L'usage est total et assumé : chaque commit du dépôt porte le trailer Co-Authored-By: Claude . Le rôle humain n'a pas été d'écrire les lignes mais de cadrer, arbitrer, éprouver et refuser.

6.2 Pourquoi, et sur quoi

MOTIF	EXEMPLE CONCRET
Écrire vite un socle sans intérêt pédagogique	parsing du labyrinthe, sérialisation de l'API, rendu canvas
Reproduire fidèlement un système documenté	les 4 personnalités et le bug d'adressage de 1980
Générer les tests	278 tests, dont ceux qui mesurent le déterminisme
Auditer	audit sécurité : 3 failles trouvées et fermées
Cadrer avant de coder	mesure du moteur comme environnement d'apprentissage

6.3 Exemples de demandes réelles

« *Mesure ce moteur comme environnement d'apprentissage par renforcement : vitesse, reproductibilité, nombre de points de décision. Dis-moi ce que ces chiffres excluent comme approche.* »

A produit le chiffre structurant du projet : 34 points de décision sur 300 cases, donc la décision par intersection, et l'exclusion du DQN sur pixels (le moteur ne tourne « qu'à » 50 parties/s, insuffisant pour 100 000 épisodes).

« *Le mouvement est saccadé quand je joue.* »

Diagnostic en deux causes cumulées : cadence et vitesse confondues dans un même réglage, et interpolation client calée sur l'arrivée des messages plutôt que sur le rythme des pas. Puis balayage mesuré du seuil de rattrapage.

« *Ce test itère sur une liste vide, prouve-le ou change de plan.* »

A mené au constat mesuré de l'absence d'impasse et au plan construit exprès.

« *Donne à l'agent la position de chaque fantôme et de la nourriture.* »

Le jeu de descripteurs `positions`, et le résultat négatif du §5.6 : moins bon de 18 %, publié tel quel.

« *Les positions des fantômes sont aléatoires ?* »

Non : une seule configuration de départ sur 100 parties, à l'entraînement comme à l'évaluation. Trois semaines de conception ne l'avaient pas vu ; une question l'a trouvé. Le §5.7 en est la réponse mesurée.

« Relis le sujet et audite tout le projet contre lui. »

Le jour du rendu : PDF manquant, CI absente, prérequis absents, critère des 70 % jamais chiffré. Quatre écarts fermés dans la journée.

6.4 Ce qui a été décidé, corrigé ou refusé côté humain

- **Le périmètre** : le sujet initial était le back-end seul ; le client web puis l'agent apprenant sont des décisions prises en cours de route.
- **L'approche d'IA** : renforcement plutôt que non-supervisé strict.
- **Les bugs trouvés en jouant, pas par les tests** : le jeu tournait à 48 cases/s (injouable) puis restait saccadé. Aucun test ne pouvait le voir, seul un humain, manette en main.
- **Les deux résultats les plus importants du rendu viennent de questions humaines** (§5.6 et §5.7), pas d'une initiative de l'outil. L'outil a mesuré ; la question qui valait la peine d'être posée est l'apport humain.
- **Ce qui a été refusé** : des élargissements de périmètre sans raison, et un test « vert » qui ne prouvait rien.

6.5 Ce que ça change pour la lecture du dépôt

Les choix structurants sont documentés à l'endroit où ils s'appliquent (docstring de `r1/environment.py`, commentaires sur les gardes de `metrics.py`), et chaque message de commit explique le *pourquoi*, pas le *quoi*. N'importe quelle ligne peut être justifiée à la demande.

7. Fichiers et données

Le projet n'a pas de jeu de données : l'apprentissage se fait par interaction avec la simulation, pas sur un corpus.

CHEMIN	CONTENU	NÉCESSAIRE POUR
<code>src/pacman/mazes/*.txt</code>	labyrinthes, un caractère par case	jouer et apprendre
<code>results/poids_*.json</code>	douze poids appris, lisibles	rejouer un agent entraîné sans le réentraîner
<code>results/campagne.json</code>	toutes les mesures du §5	vérifier les chiffres
<code>scripts/campagne_r1.py</code>	campagne complète, graines fixées	tout reproduire en une commande

7.1 Installation et lancement

```
python -m venv .venv
.venv\Scripts\activate          # Windows (source .venv/bin/activate ailleurs)
pip install -e ".[dev]"

pacman-server                   # le jeu : http://127.0.0.1:8000
pytest                          # 278 tests
pacman-rl baselines --ghosts 1 # le plancher et le plafond
python scripts/campagne_rl.py   # toute la campagne de mesure
```

Python 3.11 minimum. Le jeu et les agents n'ont **aucune dépendance** ; FastAPI et Uvicorn ne servent qu'au serveur, `markdown` qu'à produire ce document en HTML imprimable.

Regarder l'IA jouer : <http://127.0.0.1:8000/?ia=appris> une fois le serveur lancé.

7.2 Reproduire les chiffres de ce document

```
python scripts/campagne_rl.py          # ~25 min : entraîne et mesure
python scripts/comparer_descripteurs.py # ~30 min : base contre positions
python scripts/injecter_resultats.py    # réécrit le §5 et le README
python scripts/documentation_html.py    # docs/documentation.html -> Ctrl+P
python scripts/generer_presentation.py  # docs/presentation.pptx, le support oral
```

`--sans-entrainement` sur la première rejoue les mesures à partir des poids déjà présents dans `results/`, sans réapprendre : c'est aussi le contrôle que les poids publiés donnent bien les chiffres publiés.

8. Limites connues

- **L'agent ne finit pas un niveau à quatre fantômes.** C'est le plafond attendu d'un modèle linéaire à douze poids : il évalue chaque intersection isolément, sans planifier trois coups à l'avance. Il joue bien mieux que le hasard, il ne joue pas comme un humain.
- **Les features sont écrites à la main.** C'est un choix (lisibilité), mais cela veut dire que l'agent hérite de l'analyse humaine du problème : il n'apprend pas *quoi regarder*, seulement *combien ça compte*.
- **Un seul labyrinthe est mesuré.** Les poids appris devraient se transférer, puisque les features sont topologiques et non positionnelles. Le §5.7 en vérifie une moitié : déplacer les quatre fantômes ne dégrade pas la politique, mais changer de *plan* reste non mesuré : c'est une autre question, et elle demanderait un second labyrinthe.
- **Le rendu visuel n'a pas été inspecté image par image** ; il a été validé en jouant.

9. Pistes d'amélioration

- **Un réseau à la place du modèle linéaire**, pour *croiser* les descripteurs au lieu de les additionner : la limite que le §5.6 a rendue visible.
- **Une recherche à budget de simulations (MCTS)** plutôt qu'à profondeur fixe, pour adapter le coût à la difficulté de chaque intersection.
- **Un second labyrinthe**, pour mesurer le transfert des poids appris : le §5.7 n'en vérifie que la moitié.
- Features apprises plutôt qu'écrites (auto-encodeur sur l'état local) : le non-supervisé retrouverait ici sa vraie place, en amont de la politique.
- Niveaux multiples et vitesses croissantes, comme l'arcade.
- Mode multijoueur, et fantômes « chasseurs » exploitant le vrai plus court chemin (`pathfinding`) pour un mode difficile : ils cesseraient d'être myopes, donc battables.